

DEPENDENCY_MAP_PLAIN

Dependency Map — In Plain English

Plain-language companion to `docs/planning/DEPENDENCY_MAP.md`. For the exact spec, read that / its PDF.

In one sentence

This is the wiring diagram: it lists every artifact, what must exist before you can build it, the safe order to build in, and which few pieces are load-bearing enough to deserve extra care.

Why this matters to you

Building things in the wrong order wastes time — you can't compile the registry before the tables it references exist. This map gives you a guaranteed-safe order (a topological sort) and flags the keystones that, if wrong, silently break everything downstream. When you're unsure what to build next, this is the doc to open.

The big ideas, simply

- **Tiers T0–T13.** Artifacts are stacked in layers. Each one only depends on its own tier or lower. No piece depends on something built later, so the order is always safe.
- **Keystones (★).** A handful of artifacts are load-bearing — many things break if they're wrong. Build these with the most care and the most tests.
- **Parallelizable (||).** Some artifacts sit on independent branches and can be built at the same time as the main data spine.
- **The critical path is the spine.** The longest must-be-sequential chain runs: foundation → macros → staging → sessionization → funnel → `fct_funnel` → `fct_daily_funnel` → registry → MCP servers → agents → eval → CI. This chain sets the schedule.
- **A co-critical memory branch.** Because the eval runs all 7 agents, the memory tools (`save_diagnosis` / `recall_prior`) must converge before the full eval can run.

What you actually do (in order)

The map's recommended first six steps, which unblock the most with the least prerequisite:

1. **A0.1–A0.4** — repo scaffold, GCP/IAM/ADC, dbt config, and `CLAUDE.md`.
2. **A1.1–A1.6** — the four macros (especially `channel_group_case` ★), the source declaration, and the seed.
3. **A2.1–A2.3** — staging models plus tests.
4. **A3.1 → A3.2** — sessionization, then the `reached_*` funnel flags. Spend disproportionate care and tests here.
5. **A5.1** — the semantic registry, right after the funnel facts (A4.6/A4.7) exist.
6. **A6.3 stats-mcp** — build `decompose_change` in parallel from day one; it needs no data, so unit-test it early.

After M0, run three tracks at once: **Track A** the data spine (the critical path), **Track B** the math servers (`stats-mcp` , `experiment-mcp`), **Track C** memory and finance. All three converge at the agents (T8).

Easy things to get wrong

- **Letting the registry (A5.1) drift from the marts.** A column rename in a fact must update exactly one YAML; CI referential-integrity enforces it.
- **Mispointing `fct_daily_funnel`.** It aggregates `fct_funnel` (A4.6), which carries `session_revenue` — not `int_ga4_funnel_steps`. Get this wrong and you drop revenue from the daily grain and break the eval's dollar labels.
- **Skipping memory seeds.** Without the seeded `seasonality_calendar`, the Critic is blind to known confounds like Black Friday 2020 and the January trough.
- **Two copies of the registry.** A5.1 and A6.2 share one physical file: `models/semantic/semantic_layer.yaml`. A divergence silently desyncs the governed-SQL guarantee.
- **Skipping the keystone golden tests.** Sessionization, funnel monotonicity, revenue reconciliation, and `decompose_change` all fail silently — golden tests are mandatory.

Glossary — the exact words, demystified

- **Artifact** — any buildable file or component, each with an ID like A0.1 or A6.3.
- **Tier (T0–T13)** — the dependency layer an artifact lives in; lower tiers build first.
- **DAG** — directed acyclic graph; the dependency web with no loops, so a safe order always exists.

- **Critical path / spine** — the longest sequential chain; it determines the schedule.
- **Co-critical branch** — the memory chain that must finish before the full 7-agent eval can run.
- **Keystone (★)** — a load-bearing artifact (the registry, sessionization, `decompose_change`).
- **reached_* flags** — monotonic funnel flags: reaching a step or any later step.
- **fan-out** — how many other artifacts a piece unblocks; build high-fan-out pieces first.
- **[P0] – [P3/P4]** — maturity tags: P0 is MVP, P3/P4 is production/frontier.

When to open the real doc

Open `DEPENDENCY_MAP.md` (and its copy under `pdf/`) whenever you're unsure what's safe to build next, need the full tiered inventory, or want the exact critical-path chain and the continuity guardrails (§4, §6, §8).